SIGNALS AND SYSTEMS

ECA0301-LAB MANUAL EXPERIMENTS

Experiment No. 1

Generation and Classification of Continuous-Time and Discrete-Time Signals

OBJECTIVE:

(i) To generate and visualize basic continuous-time signals such as unit step, unit impulse, ramp, sinusoidal, and exponential signals.

(ii) To generate and visualize basic discrete-time signals and analyze their amplitude and time-index characteristics.

(iii) To generate and classify continuous-time and discrete-time signals as periodic or aperiodic based on their periodicity characteristics.

(iv) To decompose a given signal into its even and odd components and verify the reconstruction of the original signal.

(i) To generate and visualize basic continuous-time signals such as unit step, unit impulse, ramp, sinusoidal, and exponential signals.

Expected Outcomes

• Generate impulse, step, ramp, exponential and sinusoidal continuous-time signals.

• Obtain correctly scaled time-domain plots.

• Determine measurable parameters with preferably ≤5% deviation where applicable.

• Explain the generated waveform using its mathematical expression.

Software / Laboratory Environment

MATLAB

Input Parameter Table S.No.	**Input Parameter**	**Symbol / Setting**	**Value**	**Unit**
1	Time vector	t	−1 to 1	s
2	Sampling interval for simulation	Δt	0.001	s
3	Sinusoidal amplitude	A	2	V
4	Sinusoidal frequency	f0	5	Hz

Key Conditions & Threshold Values for Analysis:

- Use a sufficiently small simulation interval to represent the waveform accurately.
- The time axis must clearly display the relevant waveform features.
- For a sinusoidal signal, T0 = 1/f0.
- Numerical deviation for measurable parameters should preferably be ≤5%.

MATLAB Coding:

```
% Experiment 1: Generation of Continuous-Time Signals
clc; clear; close all;
t = -1:0.001:1;
A = 2; f0 = 5; a = 1;
imp = double(abs(t)<0.0005);
step = double(t>=0);
ramp = t.*step;
expSig = exp(-a*t).*step;
sinSig = A*sin(2*pi*f0*t);
figure;
signals = {imp, step, ramp, expSig, sinSig};
titles = {'Unit Impulse','Unit Step','Ramp','Exponential','Sinusoidal'};
for k = 1:5
    subplot(3,2,k);
    plot(t,signals{k});
    grid on;
    title(titles{k});
    xlabel('t');
    ylabel('Amplitude');
end
```

(ii) To generate and visualize basic discrete-time signals and analyze their amplitude and time-index characteristics.

Expected Outcomes:

• Generate correctly indexed discrete-time sequences.

• Plot sequences using stem representation.

• Identify discrete-time waveform characteristics and periodicity.

• Validate the generated samples against theoretical expressions.

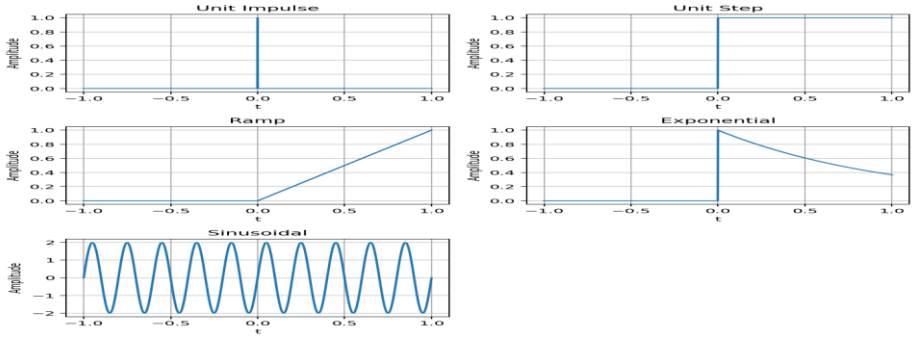
Software / Laboratory Environment:

MATLAB

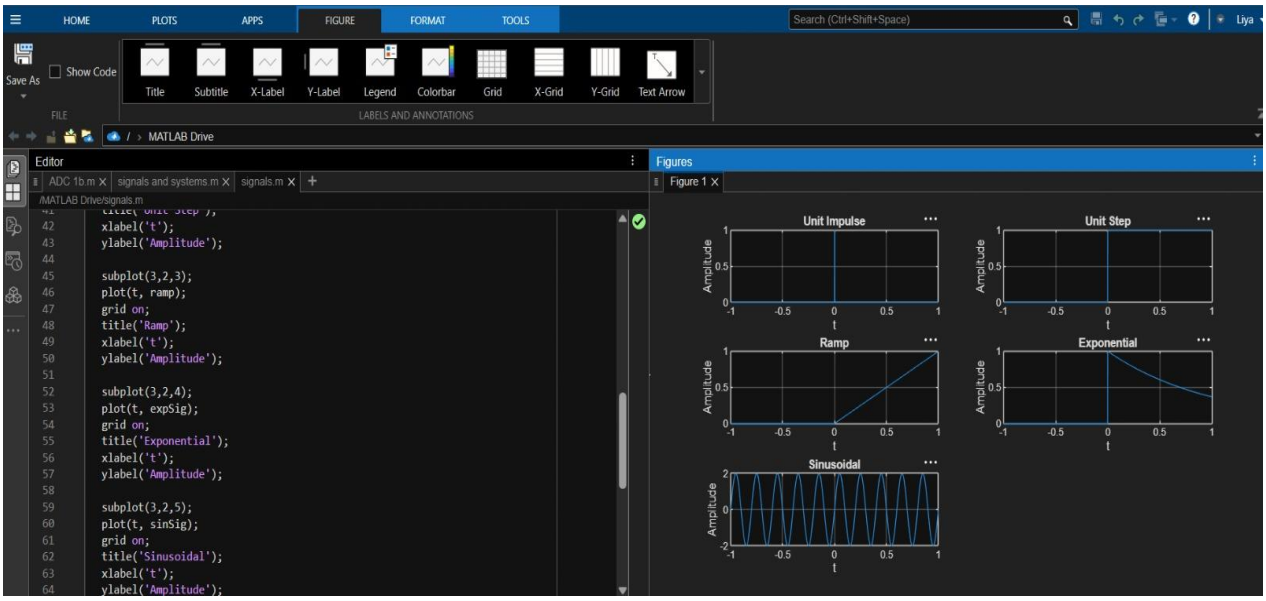
MATLAB Coding:

```
% Experiment 1: Generation of Continuous-Time Signals
clc;
clear;
close all;
t = -1:0.001:1;
A = 2;
f0 = 5;
 a = 1;
 imp = double(abs(t)<0.0005);
 step = double(t>=0);
ramp = t.*step;
expSig = exp(-a*t).*step;
sinSig = A*sin(2*pi*f0*t);
figure; subplot(3,2,1);
plot(t,imp); grid on; title('Unit Impulse'); xlabel('t'); ylabel('Amplitude'); subplot(3,2,2); plot(t,step);
 grid on; title('Unit Step'); xlabel('t'); ylabel('Amplitude'); subplot(3,2,3); plot(t,ramp);
 grid on; title('Ramp'); xlabel('t'); ylabel('Amplitude'); subplot(3,2,4); plot(t,expSig);
grid on; title('Exponential'); xlabel('t'); ylabel('Amplitude'); subplot(3,2,5); plot(t,sinSig);
grid on; title('Sinusoidal'); xlabel('t'); ylabel('Amplitude');
```

EXPECTED GRAPH:



OUTPUT GRAPH:



(ii) To generate and visualize basic discrete-time signals and analyze their amplitude and time-index characteristics.

Expected Outcomes:

• Generate correctly indexed discrete-time sequences.

• Plot sequences using stem representation.

• Identify discrete-time waveform characteristics and periodicity.

• Validate the generated samples against theoretical expressions.

Software / Laboratory Environment:

MATLAB

Input Parameter Table S.No.	Input Parameter	Symbol / Setting	Value	Unit
1	Sample index	N	−20 to 20	samples
2	Sinusoidal amplitude	A	2	V
3	Normalized frequency	f0/fs	0.1	cycles/sample
4	Exponential constant	A	0.1	per sample
Input Parameter Table S.No.	**Input Parameter**	**Symbol / Setting**	**Value**	**Unit**

1	Sample index	N	−20 to 20	samples
2	Sinusoidal amplitude	A	2	V
3	Normalized frequency	f0/fs	0.1	cycles/sample
4	Exponential constant	A	0.1	per sample

Formulae / Analysis:

$x[n] = A \sin(2\pi f_0 n + \varphi)$.

Periodic DT signal: $x[n + N_0] = x[n]$ for a positive integer N_0.

$x[n] = a^n u[n]$ for a suitable discrete exponential sequence.

Even sequence: $x[n] = x[-n]$; odd sequence: $x[n] = -x[-n]$.

Experimental Procedure:

1. Define the integer sample index n.
2. Generate unit impulse, unit step, ramp, exponential and sinusoidal sequences.
3. Display each sequence using a stem plot.
4. Determine sample amplitudes and period where applicable.
5. Classify the sequence as periodic/non-periodic and even/odd wherever applicable.
6. Compare the generated samples with theoretical values and record observations.

Key Conditions & Threshold Values for Analysis:

- Discrete-time signals are defined only at integer indices.
- Stem plots should be used to emphasize discrete samples.
- DT sinusoidal periodicity depends on the normalized frequency satisfying the rationality condition.
- Use sufficient sample range to observe the sequence behavior.

MATLAB Coding:

```
% Experiment 2: Generation of Discrete-Time Signals

clc;

clear;

close all;

n = -20:20;

A = 2;
```

```
f0 = 0.1;
a = 0.1;
% Unit Impulse Signal
imp = double(n == 0);
% Unit Step Signal
step = double(n >= 0);
% Ramp Signal
ramp = n .* step;
% Exponential Signal
expSig = (0.8).^n .* step;
% Discrete-Time Sinusoidal Signal
sinSig = A * sin(2 * pi * f0 * n);
% Plot all signals
figure;
subplot(3,2,1);
stem(n, imp, 'filled');
grid on;
title('Unit Impulse');
subplot(3,2,2);
stem(n, step, 'filled').;
grid on;
title('Unit Step');
subplot(3,2,3);
stem(n, ramp, 'filled');
grid on;
title('Ramp');
subplot(3,2,4);
stem(n, expSig, 'filled');
grid on;
title('Exponential');
subplot(3,2,5);
```

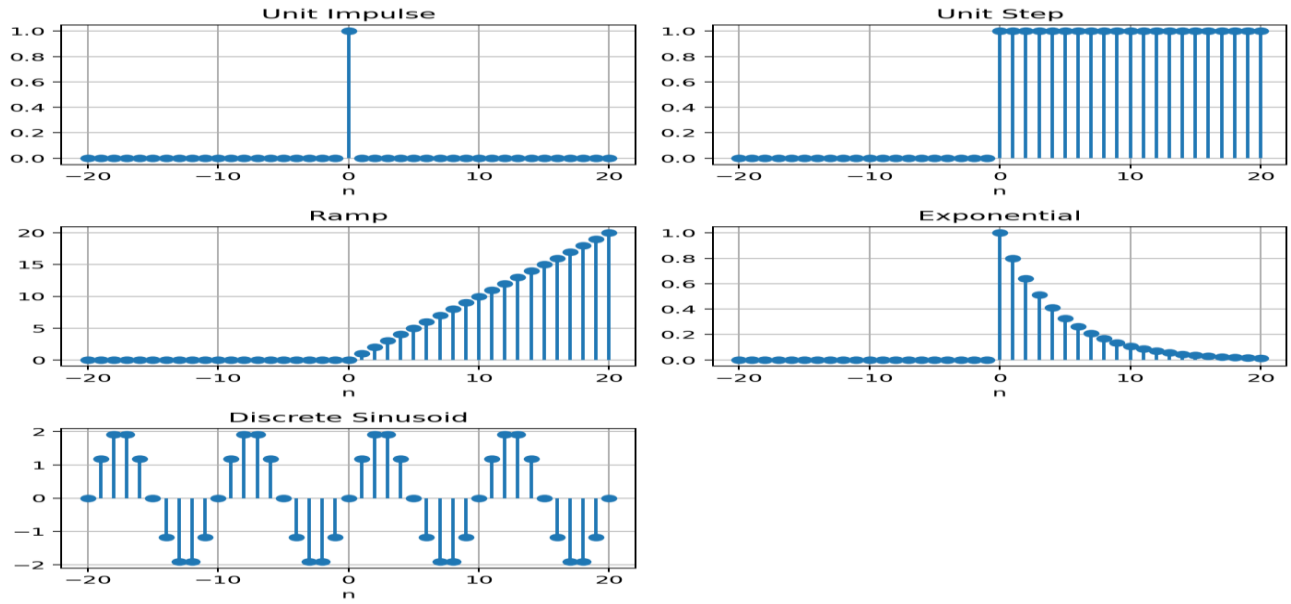
stem(n, sinSig, 'filled');

grid on;

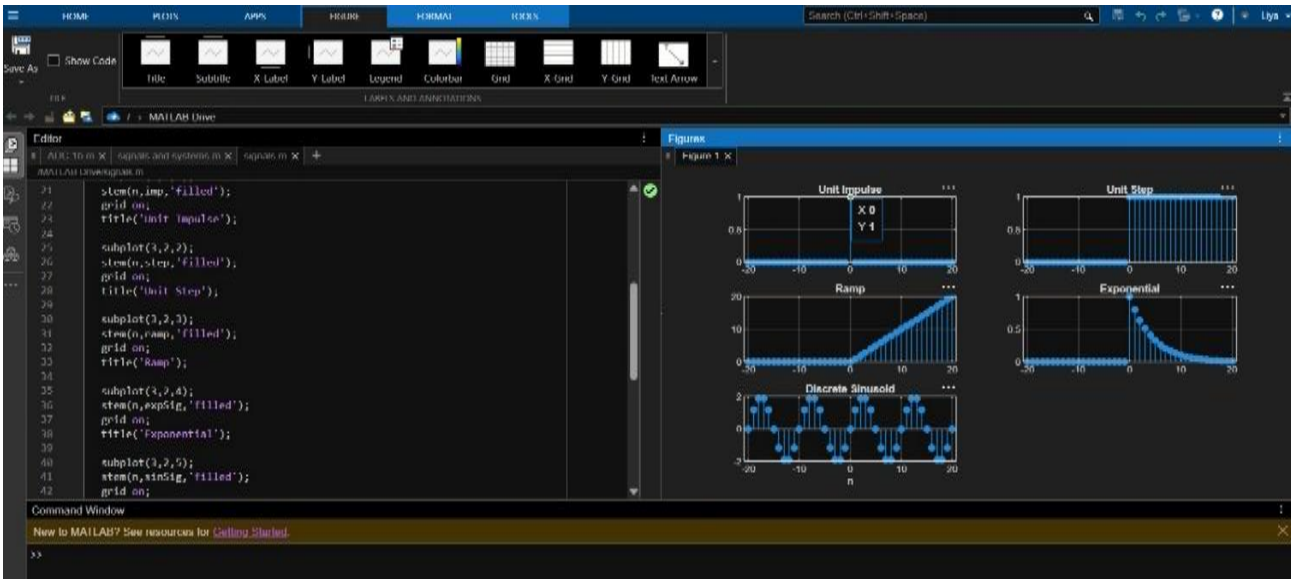
title('Discrete Sinusoid');

xlabel('n');

EXPECTED GRAPHS:



OUTPUT GRAPH:



Conclusion:

The continuous-time and discrete-time signals were generated and analyzed successfully. The signals were classified as periodic or aperiodic, and the even and odd components were obtained and used to reconstruct the original signal successfully.

Experiment No. 2

Signal Operations and Transformations

OBJECTIVE:

(i) To perform amplitude scaling and time scaling of continuous-time and discrete-time signals and analyze their effects on signal characteristics.

(ii) To perform time shifting and time reversal of continuous-time and discrete-time signals and analyze the resulting transformations.

(iii) To perform addition and multiplication of two discrete-time sequences and analyze the resulting signal characteristics.

(iv) To visualize and compare the original and transformed signals using MATLAB and interpret the effects of different signal operations.

(i) To perform amplitude scaling and time scaling of continuous-time and discrete-time signals and analyze their effects on signal characteristics.

Expected Outcomes:

- Correctly implement amplitude scaling.
- Perform time compression and time expansion.
- Measure the resulting amplitude and period.
- Explain the observed changes using the scaling equations.

Software / Laboratory Environment:

MATLAB

Input Parameter Table S.No.	**Input Parameter**	**Symbol / Setting**	**Value**	**Unit**
1	Base signal	x(t)	$\sin(2\pi 5t)$	—
2	Amplitude scaling factor	A	2	—
3	Time compression factor	a	2	—
4	Time expansion factor	a	2	—
5	Time vector	t	−1 to 1	s

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Amplitude of original	1	Record value	≤5%
2	Amplitude after scaling	2	Record value	≤5%
3	Original period	0.2 s	Measure from plot	≤5%
4	Compressed period	0.1 s	Measure from plot	≤5%

Formulae / Analysis:

Amplitude scaling: $y(t) = A\,x(t)$.

Time compression: $y(t) = x(at)$, $a > 1$.

Time expansion: $y(t) = x(t/a)$, $a > 1$.

For $x(t) = \sin(2\pi f_0 t)$, $T_0 = 1/f_0$.

Percentage deviation = |Practical − Theoretical| / |Theoretical| × 100.

Experimental Procedure:

1. Generate the base signal $x(t) = \sin(2\pi 5t)$.
2. Apply amplitude scaling using $A = 2$.
3. Apply time compression using $x(2t)$.
4. Apply time expansion using $x(t/2)$.
5. Plot the original and transformed signals using common axes.
6. Measure the amplitude and period of the transformed signals.
7. Compare the measured results with the theoretical scaling relationships.

Key Conditions & Threshold Values for Analysis:

- Use identical axes when comparing waveforms.
- Amplitude scaling should not change the time period.
- Time compression decreases the period, while time expansion increases the period.
- Numerical parameter deviation should preferably be ≤5%.

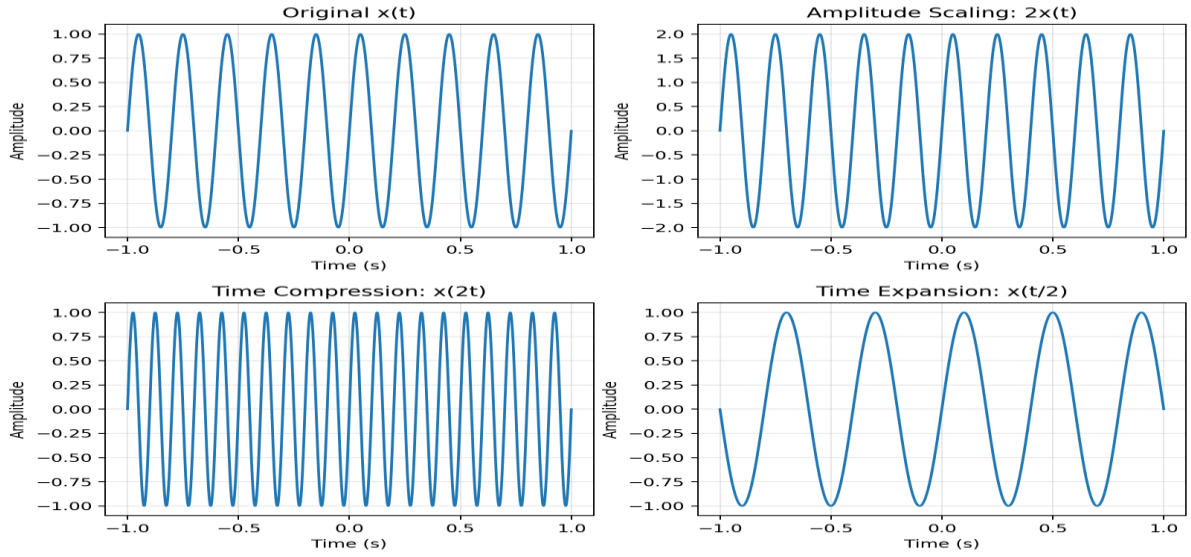
MATLAB Coding:

% Experiment 2(i): Amplitude and Time Scaling

```
clc;
clear;
close all;
t = -1:0.001:1;
% Original Signal
x = sin(2*pi*5*t);
% Amplitude Scaling
yA = 2*x;
% Time Compression
yC = sin(2*pi*5*(2*t));
% Time Expansion
yE = sin(2*pi*5*(t/2));
% Plot the signals
figure;
subplot(2,2,1);
plot(t, x);
grid on;
title('Original x(t)');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(2,2,2);
plot(t, yA);
grid on;
title('Amplitude Scaling: 2x(t)');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(2,2,3);
plot(t, yC);
grid on;
title('Time Compression: x(2t)');
xlabel('Time (s)');
```

```
ylabel('Amplitude');
subplot(2,2,4);
plot(t, yE);
grid on;
title('Time Expansion: x(t/2)');
xlabel('Time (s)');
ylabel('Amplitude');
```

Input and Output Graphs – Amplitude and Time Scaling:





(iii) To perform addition and multiplication of two discrete-time sequences and analyze the resulting signal characteristics

Expected Outcomes:

- Obtain correct sum and product sequences.

- Preserve the common sample index for aligned sequences.
- Verify representative output samples numerically.
- Interpret how addition and multiplication change sequence values.

Software / Laboratory Environment:

MATLAB

Input Parameter Table S.No.	Input Parameter	Symbol / Setting	Value	Unit
1	First sequence	x[n]	{1, 2, 1}	—
2	Second sequence	y[n]	{1, 1, 2}	—
3	Index range	n	0 to 2	samples

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Sum sequence	x[n]+y[n] = {2,3,3}	Record sequence	Exact
2	Product sequence	x[n]y[n] = {1,2,2}	Record sequence	Exact
3	Maximum sum	3	Record value	Exact
4	Sequence length	3	Record value	Exact

Formulae / Analysis:

$z[n] = x[n] + y[n]$.

$p[n] = x[n]y[n]$.

Operations are performed for corresponding sample indices.

Percentage error, where applicable = $|\text{Measured} - \text{Theoretical}| / |\text{Theoretical}| \times 100$.

Experimental Procedure:

Define the sample index n.

1. Enter x[n] and y[n] with aligned indices.
2. Compute the sum sequence.
3. Compute the product sequence using pointwise multiplication.

4. Plot the input and resulting sequences using stem plots.

5. Compare the calculated samples with manual/theoretical results.

Key Conditions & Threshold Values for Analysis:

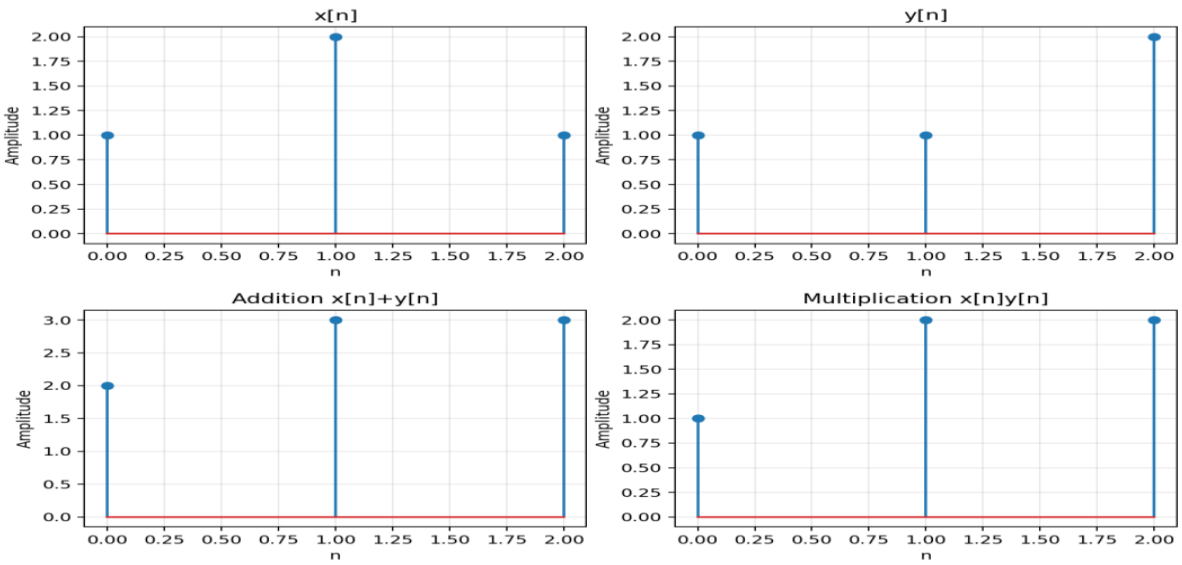
- Both sequences must use a clearly defined and aligned index set.
- Do not confuse pointwise multiplication with convolution.
- Use sufficient plotting range to show all samples.
- Numerical results should match the theoretical calculations.

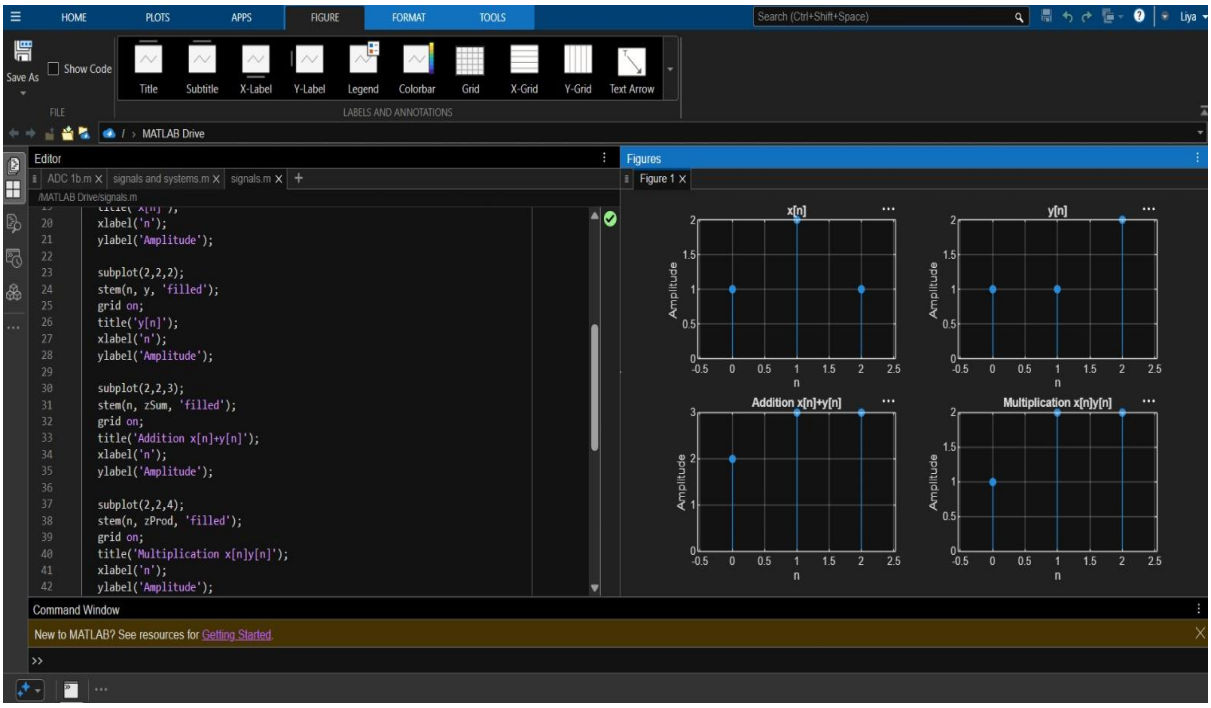
MATLAB Coding:

```
% Experiment 2(iii): Addition and Multiplication of Two Sequences
clc;
clear;
close all;
n = 0:2;
x = [1 2 1];
y = [1 1 2];
zSum = x + y;
zProd = x .* y;
figure;
subplot(2,2,1);
stem(n, x, 'filled');
grid on;
title('x[n]');
xlabel('n');
ylabel('Amplitude');
subplot(2,2,2);
stem(n, y, 'filled');
grid on;
title('y[n]');
xlabel('n');
ylabel('Amplitude');
subplot(2,2,3);
```

```
stem(n, zSum, 'filled');
grid on;
title('Addition x[n]+y[n]');
xlabel('n');
ylabel('Amplitude');
subplot(2,2,4);
stem(n, zProd, 'filled');
grid on;
title('Multiplication x[n]y[n]');
xlabel('n');
ylabel('Amplitude');
```

Input and Output Graphs – Addition and Multiplication of Two Sequences:





Conclusion:

Amplitude and time scaling, time shifting, time reversal, sequence addition and sequence multiplication were successfully implemented. The original and transformed signals were compared, and the observed waveform changes agreed with the corresponding theoretical signal-operation relationships

Experiment 3

Energy and Power Analysis of Signals

OBJECTIVE:

To calculate the energy and average power of given continuous-time and discrete-time signals and classify them as energy signals, power signals, or neither based on their energy and power characteristics.

Expected Outcomes:

- Calculate finite energy for an energy-type signal.
- Calculate average power for a periodic/power-type signal.
- Classify the signal based on the obtained values.
- Obtain numerical results consistent with theoretical expectations.

Software / Laboratory Environment:

MATLAB

Input Parameter Table S.No.	Input Parameter	Symbol / Setting	Value	Unit
1	Signal 1	$x_1(t)$	e^(−t)u(t)	—
2	Signal 2	$x_2(t)$	sin(2π5t)	—
3	Integration interval	t	0 to 5	s
4	Period for power calculation	T_0	0.2	s

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Energy of $x_1(t)$	$\int\|x_1(t)\|^2dt = 0.5$	Record numerical result	Compare with theory
2	Average power of $x_2(t)$	1/2 for unit-amplitude sinusoid	Record numerical result	Preferably ≤5%
3	Signal classification	Energy / Power	Record classification	Correct

Formulae / Analysis:

$E = \int_{-\infty}^{\infty} |x(t)|^2 \, dt$ for a continuous-time signal.

$P = \lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |x(t)|^2 \, dt$.

For $x(t) = \sin(2\pi f_0 t)$, average power is 1/2 for unit amplitude.

A signal with finite non-zero energy and zero average power is an energy signal; a finite non-zero average-power signal is a power signal.

Percentage deviation = |Practical − Theoretical| / |Theoretical| × 100.

Experimental Procedure:

1. Define the time vector and generate the selected signals.

1. Compute the squared magnitude of each signal.

2. Numerically integrate over a sufficiently large interval for energy.

3. Compute average power over an interval containing several periods for the periodic signal.

4. Compare numerical results with analytical expectations.

5. Classify the signals and record observations.

Key Conditions & Threshold Values for Analysis:

- The integration interval should be sufficiently long for decaying signals.
- For periodic signals, use an interval containing complete cycles where practical.
- Use adequate sampling resolution for numerical integration.
- Energy and power definitions must not be interchanged.
- For $x_1(t) = e^{(-t)}u(t)$, the theoretical energy is finite and equal to 0.5.
- For $x_2(t) = \sin(2\pi 5t)$, the theoretical average power is 0.5.

MATLAB Coding:

```
% Experiment 3: Energy and Power Analysis

clc; clear; close all;

dt = 0.001;

t = 0:dt:5;

xE = exp(-t);

xP = sin(2*pi*5*t);

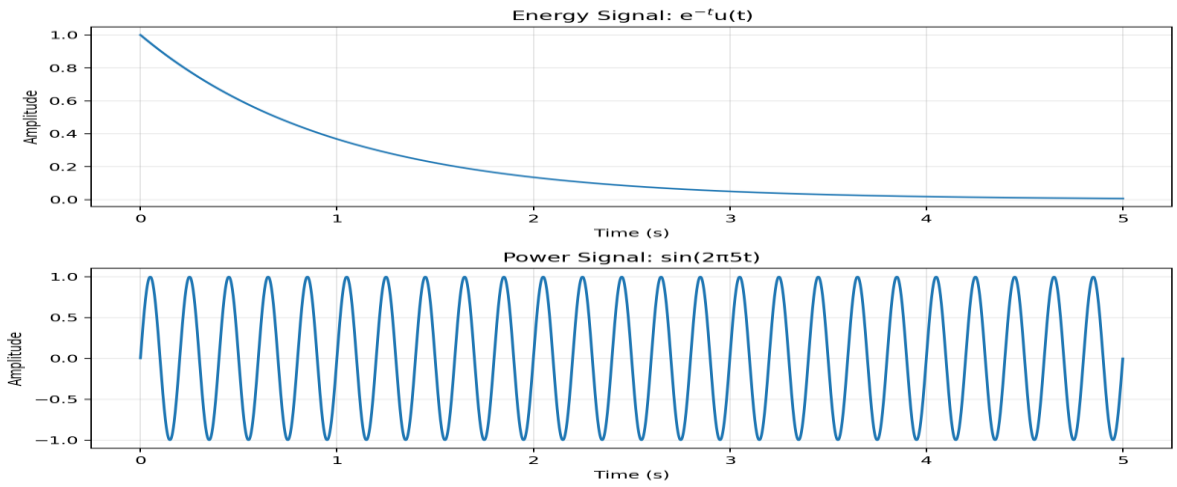
E_num = trapz(t,abs(xE).^2);

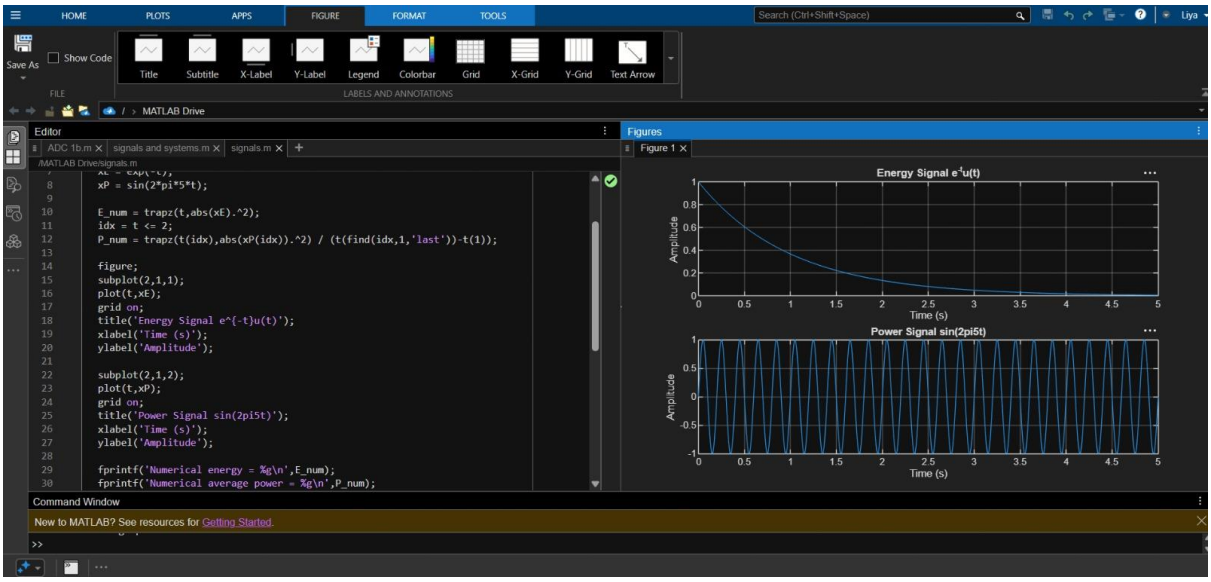
idx = t <= 2;

P_num = trapz(t(idx),abs(xP(idx)).^2) / (t(find(idx,1,'last'))-t(1));
```

```
figure;
subplot(2,1,1);
plot(t,xE);
grid on;
title('Energy Signal e^{-t}u(t)');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(2,1,2);
plot(t,xP);
grid on;
title('Power Signal sin(2pi5t)');
xlabel('Time (s)');
ylabel('Amplitude');
fprintf('Numerical energy = %g\n',E_num);
fprintf('Numerical average power = %g\n',P_num);
```

Input and Output Graphs – Energy and Power Analysis:





Observation / Result:

The numerical energy of $x_1(t) = e^{(-t)}\, u(t)$ is approximately 0.5000, and the numerical average power of $x_2(t) = \sin(2\pi 5t)$ is approximately 0.5000. The results agree with the theoretical values within the expected numerical accuracy.

Conclusion:

Energy and average power were calculated successfully, and the given signals were classified as energy and power signals based on their energy and power characteristics.

Experiment – 4:

Verification of System Properties: Linearity, Time Invariance, Causality and Stability

Objective:

i) To verify the linearity and time-invariance properties of continuous-time and discrete-time systems using suitable input signals and system responses.

(ii) To analyze and classify continuous-time and discrete-time systems as causal or non-causal based on their input-output relationships.

(iii) To analyze and classify systems as memoryless or dynamic based on their dependence on present and past input values.

(iv) To verify the BIBO stability of given systems and classify them as stable or unstable for real-time automation applications.

i) To verify the linearity and time-invariance properties of continuous-time and discrete-time systems using suitable input signals and system responses.

Expected Outcomes:

• Determine whether the selected system satisfies homogeneity and additivity.

• Verify whether delaying the input produces the corresponding output delay.

• Distinguish linear/time-invariant and non-linear/time-varying behavior.

• Support the conclusion using plots and numerical error.

Software / Laboratory Environment:

MATLAB

Input Parameter Table S.No.	**Input Parameter**	**Symbol / Setting**	**Value**	**Unit**
1	Test system	CT example	$y(t)=2x(t)$	—
2	Input 1	x1(t)	$\sin(2\pi 5t)$	—
3	Input 2	x2(t)	$\cos(2\pi 5t)$	—
4	Scalar	a	2	—
5	Time shift	t0	0.2	s

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Superposition error	y[a x1+b x2] − [a y1+b y2]	Record error	Preferably ≈0
2	Homogeneity error	y[ax] − ay[x]	Record error	Preferably ≈0
3	Time-invariance error	T{x(t−t0)} − y(t−t0)	Record error	Preferably ≈0
4	Classification	Linear / Nonlinear; TI / TV	Record result	Correct

Formulae / Analysis:

Linearity: $T\{a\,x1 + b\,x2\}=aT\{x1\}+bT\{x2\}$.

Homogeneity: $T\{a\,x\}=aT\{x\}$.

Additivity: $T\{x1+x2\}=T\{x1\}+T\{x2\}$.

Time invariance: if $y(t)=T\{x(t)\}$, then $T\{x(t-t0)\}=y(t-t0)$.

Experimental Procedure:

1. Select a representative CT or DT system.
2. Generate two independent test inputs.
3. Compute the individual system outputs.
4. Form a combined input and compare its system output with the corresponding linear combination of outputs.
5. Delay the input by t0 and compare the system response with the correspondingly delayed original output.
6. Calculate numerical differences and classify the system.

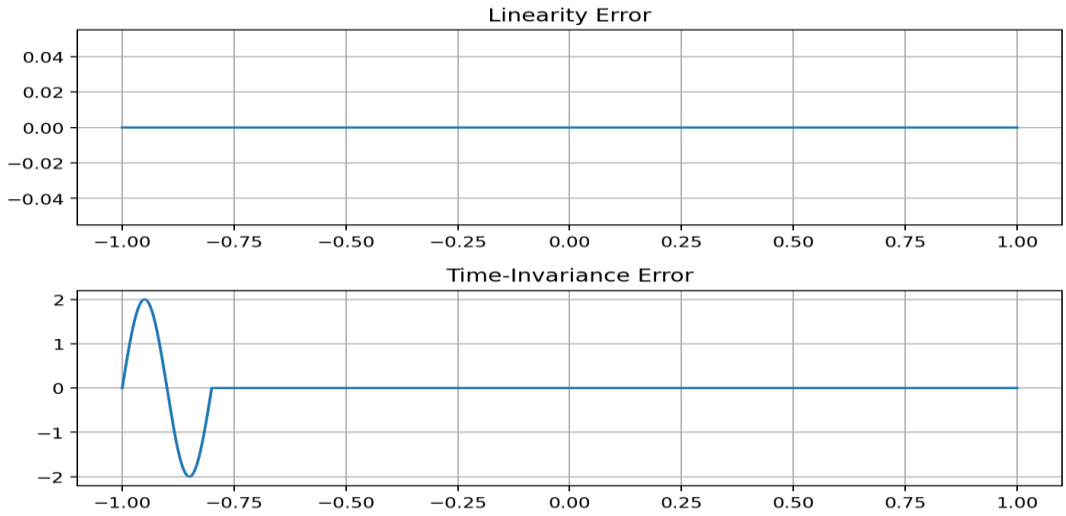
Key Conditions & Threshold Values for Analysis:

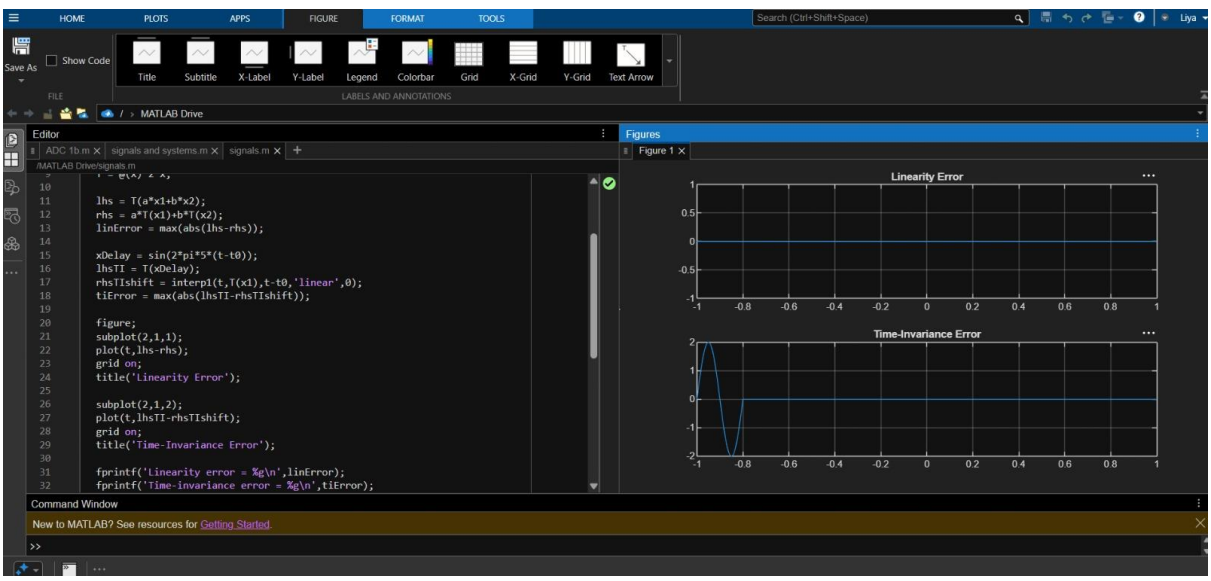
- Use identical time/sample grids for all comparisons.
- Use the same scaling coefficients in the theoretical and practical tests.
- Allow only small numerical differences due to floating-point computation.
- The classification must be based on the stated mathematical conditions.

MATLAB Coding:

```
% Experiment 11: Linearity and Time Invariance
clc; clear; close all;
t = -1:0.001:1;
x1 = sin(2*pi*5*t);
x2 = cos(2*pi*5*t);
a = 2; b = 3; t0 = 0.2;
T = @(x) 2*x;
lhs = T(a*x1+b*x2);
rhs = a*T(x1)+b*T(x2);
linError = max(abs(lhs-rhs));
xDelay = sin(2*pi*5*(t-t0));
lhsTI = T(xDelay);
rhsTIshift = interp1(t,T(x1),t-t0,'linear',0);
tiError = max(abs(lhsTI-rhsTIshift));
figure;
subplot(2,1,1);
plot(t,lhs-rhs);
grid on;
title('Linearity Error');
subplot(2,1,2);
plot(t,lhsTI-rhsTIshift);
grid on;
title('Time-Invariance Error');
fprintf('Linearity error = %g\n',linError);
fprintf('Time-invariance error = %g\n',tiError);
```

Input and Output Graphs:





Conclusion:

The system properties were successfully verified and classified. Linearity, time invariance, causality, memory, and stability were analyzed. The results confirmed the system behavior for practical applications.

Experiment 5

Fourier Series Analysis of Periodic Signals

Objective:

i) To analyze Fourier series of continuous-time periodic signals and analyze their waveform characteristics. (ii) To determine the Fourier coefficients and harmonic components of periodic signals. (iii) To generate and analyze the amplitude and phase spectra of periodic signals. (iv) To reconstruct periodic signals using Fourier series and verify the reconstructed waveform with the original signal.

i) To analyze Fourier series of continuous-time periodic signals and analyze their waveform Characteristics.

Expected Outcomes:

• Obtain the Fourier series representation of the selected periodic signal.

• Compute the required Fourier coefficients numerically.

• Plot the original and reconstructed signals for comparison.

• Explain the effect of harmonic components on reconstruction accuracy.

Software / Laboratory Environment:

MATLAB

S.No.	Input Parameter	Symbol / Setting	Value
1	Signal amplitude A	1	—
2	Fundamental frequency f0	1	Hz
3	Fundamental period T0	1	s
4	Number of harmonics N	10	—

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Fourier coefficients	Computed values	Record values	Numerical comparison
2	Original waveform	Half-wave rectified sinusoid	Record plot	Qualitative match
3	Reconstructed waveform	Fourier-series approximation	Record plot	Qualitative match
4	Harmonic count	10	Record value	Exact

Formulae / Analysis:

Fourier series: $x(t) = a0/2 + \Sigma[an \cos(n\omega 0t) + bn \sin(n\omega 0t)]$.

$\omega 0 = 2\pi f0$ and $T0 = 1/f0$.

$an = (2/T0)\int x(t)\cos(n\omega 0t)dt$; $bn = (2/T0)\int x(t)\sin(n\omega 0t)dt$.

Experimental Procedure:

1. Define the time vector and fundamental frequency.
2. Generate the half-wave rectified sinusoidal periodic signal.
3. Compute the Fourier coefficients up to N = 10.
4. Construct the Fourier-series approximation.
5. Plot the original and reconstructed signals and compare them.

Key Conditions & Threshold Values for Analysis:

- Use sufficient time resolution to represent the periodic waveform.
- Use an adequate number of harmonics for reconstruction.
- The reconstructed waveform should approach the original waveform as harmonics are included

MATLAB Coding:

```
% 5(i) Analysis of Fourier Series Representation

clc; clear; close all;

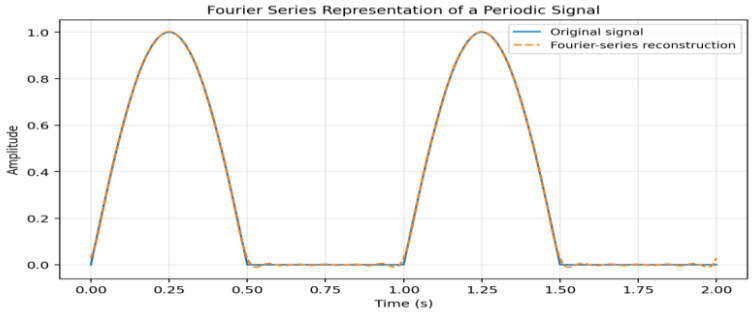
A = 1; f0 = 1; T = 1/f0;

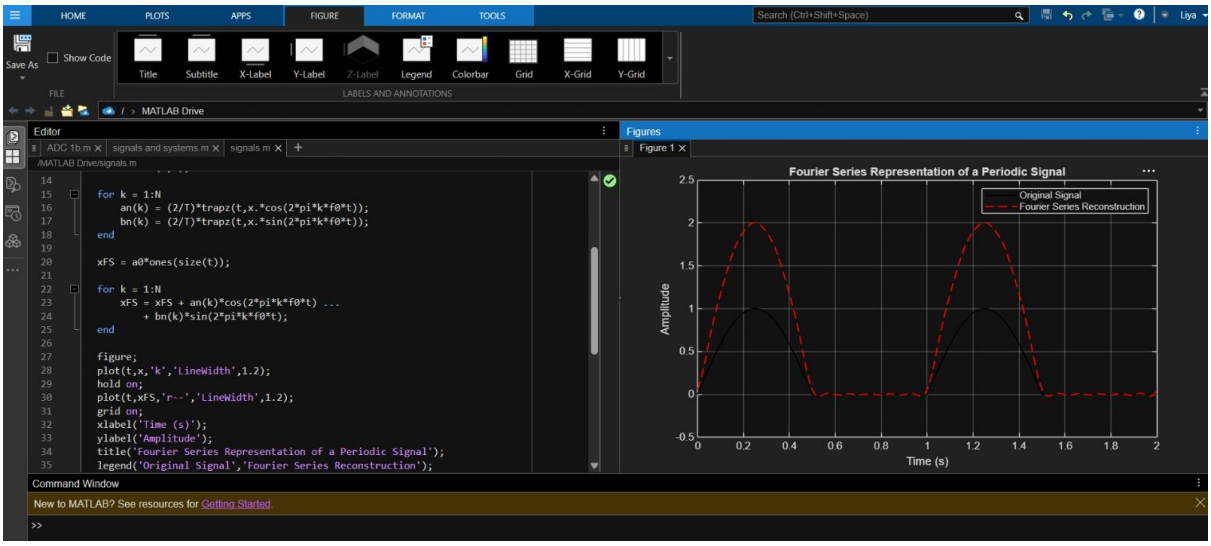
t = linspace(0,2*T,2000);

x = A*sin(2*pi*f0*t);
```

```
x(x<0) = 0;
N = 10;
a0 = (2/T)*trapz(t,x)/2;
an = zeros(1,N);
bn = zeros(1,N);
for k = 1:N
    an(k) = (2/T)*trapz(t,x.*cos(2*pi*k*f0*t));
    bn(k) = (2/T)*trapz(t,x.*sin(2*pi*k*f0*t));
end
xFS = a0*ones(size(t));
for k = 1:N
    xFS = xFS + an(k)*cos(2*pi*k*f0*t) ...
    + bn(k)*sin(2*pi*k*f0*t);
end
figure;
plot(t,x,'k','LineWidth',1.2);
hold on;
plot(t,xFS,'r--','LineWidth',1.2);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Fourier Series Representation of a Periodic Signal');
legend('Original Signal','Fourier Series Reconstruction');
```

Input and Output Graphs:





(iv) To reconstruct periodic signals using Fourier series and verify the reconstructed waveform with the original signal.

Expected Outcome:

• Reconstruct the periodic waveform using the calculated coefficients.

• Plot original and reconstructed signals together.

• Obtain the reconstruction error.

• Assess the quality of the Fourier-series approximation.

Software / Laboratory Environment:

MATLAB

Table S.No.	**Input Parameter**	**Symbol / Setting**	**Value**
1	Fundamental frequency f0	1	Hz
2	Number of harmonics N	10	—
3	Time duration	2T0	s
4	Signal	Half-wave rectified sinusoid	—

Output Parameter Table S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Reconstructed signal	Fourier approximation	Record plot	Qualitative match
2	Reconstruction error	x(t) − xr(t)	Record plot	Small error
3	Harmonic count	10	Record value	Exact
4	Signal period	1 s	Measure/verify	≤5%

Formulae / Analysis:

Reconstructed signal: $x_r(t) = a_0/2 + \Sigma[a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t)]$.

Reconstruction error: $e(t) = x(t) - x_r(t)$.

Percentage deviation = |Practical − Theoretical|/|Theoretical| × 100.

Experimental Procedure:

1. Generate the periodic signal over two periods.
2. Compute Fourier coefficients from one period.
3. Construct the reconstructed signal using 10 harmonics.
4. Calculate the error signal.
5. Plot the original, reconstructed and error signals.

Key Conditions & Threshold Values for Analysis:

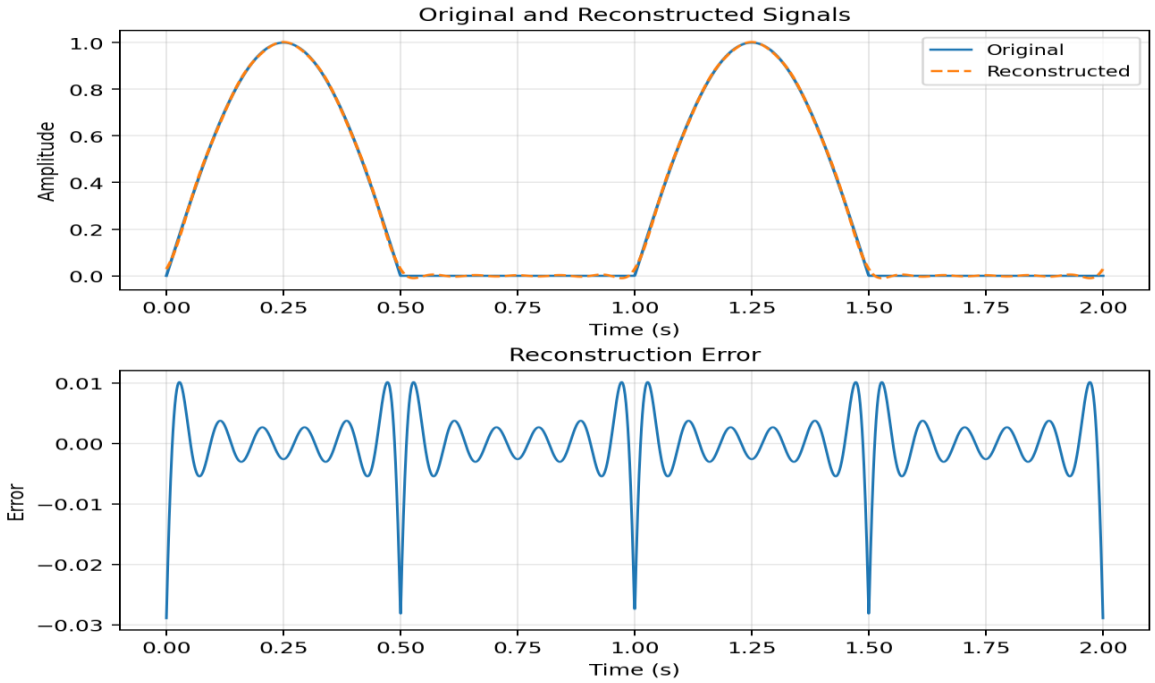
- Coefficient calculation should use one complete period.
- The reconstruction should preserve the signal periodicity.
- Error should be interpreted around discontinuities and waveform transitions.

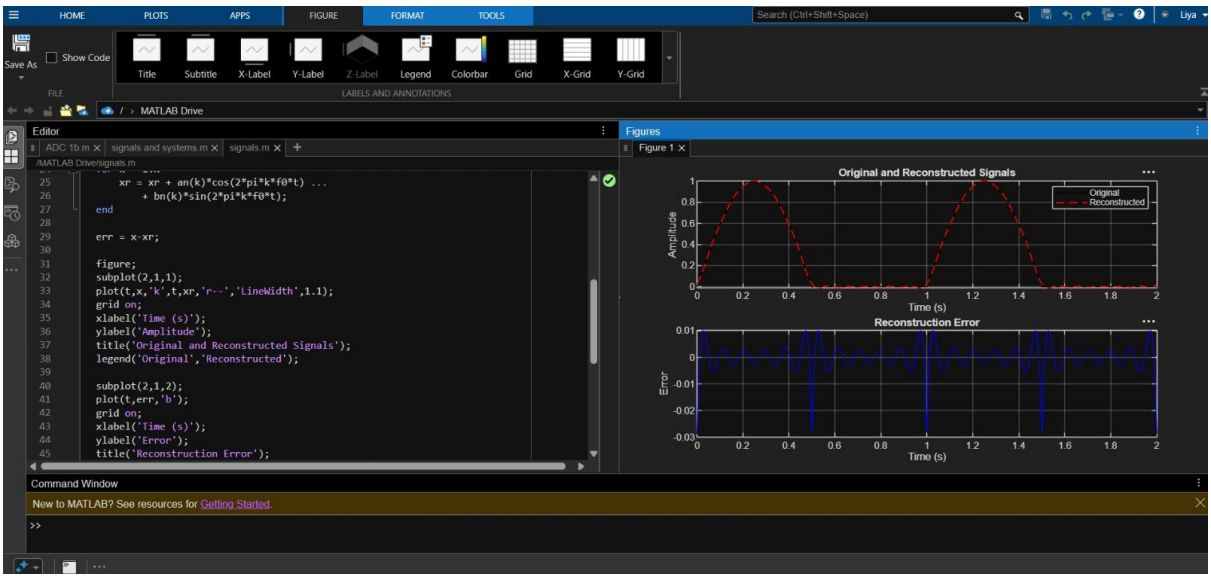
MATLAB Coding:

```
% 5(iv) Verification of Fourier Series Reconstruction and Signal Characteristics
clc; clear; close all;
f0 = 1;
T = 1/f0;
t = linspace(0,2*T,4000);
x = max(sin(2*pi*f0*t),0);
N = 10;
```

```
tt = t(1:2000);
xx = x(1:2000);
a0 = (2/T)*trapz(tt,xx);
an = zeros(1,N);
bn = zeros(1,N);
for k = 1:N
    an(k) = (2/T)*trapz(tt,xx.*cos(2*pi*k*f0*tt));
    bn(k) = (2/T)*trapz(tt,xx.*sin(2*pi*k*f0*tt));
end
xr = a0/2*ones(size(t));
for k = 1:N
    xr = xr + an(k)*cos(2*pi*k*f0*t) ...
      + bn(k)*sin(2*pi*k*f0*t);
end
err = x-xr;
figure;
subplot(2,1,1);
plot(t,x,'k',t,xr,'r--','LineWidth',1.1);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Original and Reconstructed Signals');
legend('Original','Reconstructed');
subplot(2,1,2);
plot(t,err,'b');
grid on;
xlabel('Time (s)');
ylabel('Error');
title('Reconstruction Error');
```

Input and Output Graphs:





Conclusion

The Fourier series representation of periodic continuous-time signals was successfully analysed and the Fourier coefficients and harmonic components were determined. The amplitude and phase spectra were generated and their characteristics were studied. The periodic signal was reconstructed using Fourier series and the reconstructed waveform was verified with the original signal.

Experiment – 6

PARSEVAL'S RELATION AND HARMONIC SPECTRUM ANALYSIS

Objective:

(i)To verify Parseval's relation for continuous-time periodic signals and analyze the relationship between time-domain and frequency-domain power.

(ii) To analyze the distribution of signal power among the fundamental and harmonic components of periodic signals.

(iii) To determine and analyze the harmonic spectrum of continuous-time periodic signals.

(iv) To compare the total power obtained in the time domain with the harmonic power distribution in the frequency domain.

(i) To verify Parseval's relation for continuous-time periodic signals and analyze the relationship between time-domain and frequency-domain power

Expected Outcomes:

• Obtain time-domain average power.

• Obtain frequency-domain power using Fourier coefficients.

• Calculate the absolute error between the two results.

• Verify numerical agreement between the two domains.

Software / Laboratory Environment:

MATLAB

S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Time-domain power	Numerical integration	Record value	Compare
2	Frequency-domain power	Parseval calculation	Record value	Compare
3	Absolute error	|Ptime − Pfreq|	Record value	Small
4	Signal period	1 s	Record value	≤5%

Formulae / Analysis:

$P = (1/T0)\int 0^{T0} |x(t)|^2 dt.$

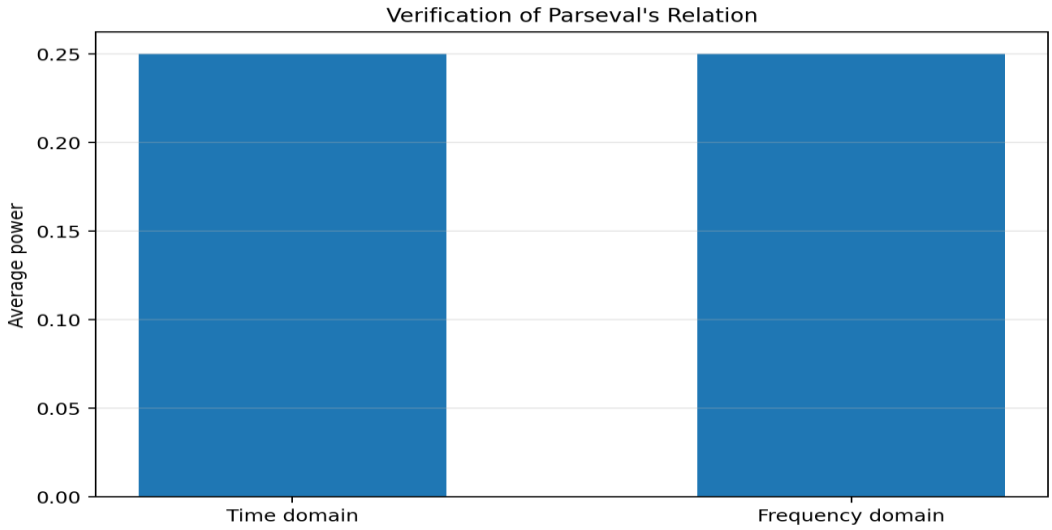
Parseval relation: $P = a0^2/4 + 1/2\ \Sigma(an^2 + bn^2)$.

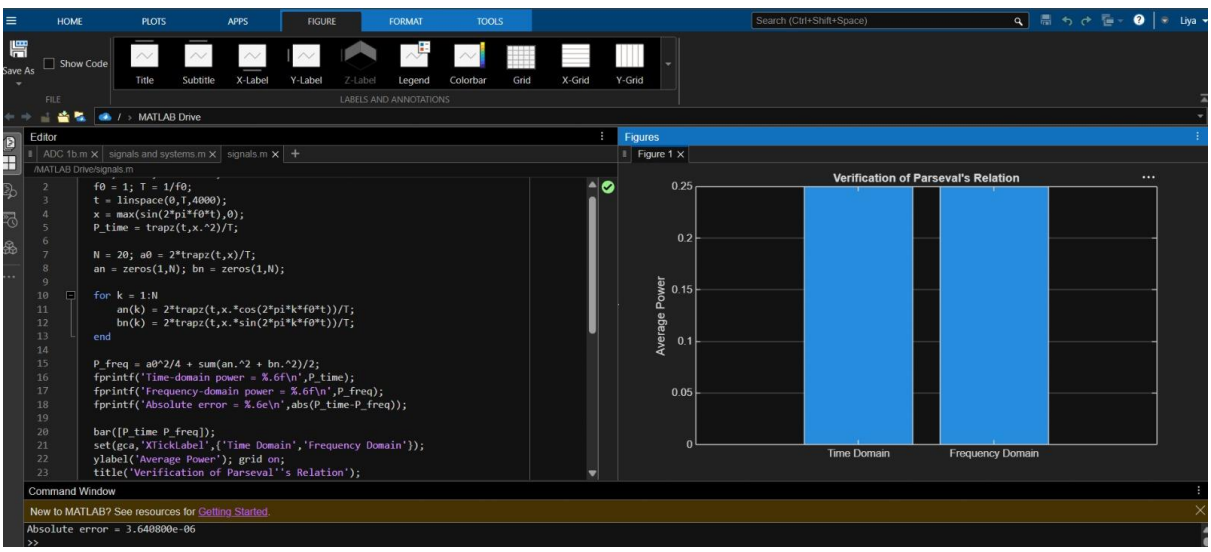
Absolute error = |Ptime − Pfreq|.

MATLAB Coding:

```
clc; clear; close all;
f0 = 1; T = 1/f0;
t = linspace(0,T,4000);
x = max(sin(2*pi*f0*t),0);
P_time = trapz(t,x.^2)/T;
N = 20; a0 = 2*trapz(t,x)/T;
an = zeros(1,N); bn = zeros(1,N);
for k = 1:N
    an(k) = 2*trapz(t,x.*cos(2*pi*k*f0*t))/T;
    bn(k) = 2*trapz(t,x.*sin(2*pi*k*f0*t))/T;
end
P_freq = a0^2/4 + sum(an.^2 + bn.^2)/2;
fprintf('Time-domain power = %.6f\n',P_time);
fprintf('Frequency-domain power = %.6f\n',P_freq);
fprintf('Absolute error = %.6e\n',abs(P_time-P_freq));
bar([P_time P_freq]);
set(gca,'XTickLabel',{'Time Domain','Frequency Domain'});
ylabel('Average Power'); grid on;
title('Verification of Parseval''s Relation');
```

Input and Output Graphs:





Experimental Procedure:

- Generate the periodic signal over one period.
- Compute time-domain average power.
- Calculate Fourier coefficients up to N = 20.
- Evaluate frequency-domain power using Parseval's relation.
- Compare the two values and calculate absolute error.

Key Conditions & Threshold Values for Analysis:

- Use one complete period for average-power calculation.
- Use sufficient harmonics for close numerical agreement.
- Report the absolute difference between the two power calculations.

(iv) To compare the total power obtained in the time domain with the harmonic power distribution in the frequency domain.

Expected Outcomes:

• Calculate time-domain power.

• Calculate DC and harmonic power components.

• Obtain total frequency-domain power.

• Compare time-domain and frequency-domain results.

Software / Laboratory Environment:

MATLAB

S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Time-domain power	Numerical integration	Record value	Compare
2	DC power	$a0^2/4$	Record value	Numerical
3	Harmonic power	$0.5\Sigma(an^2+bn^2)$	Record value	Numerical
4	Frequency-domain power	PDC + Pharmonics	Record value	Compare

Formulae / Analysis:

Pfrequency = PDC + Pharmonics.

PDC = $a0^2/4$.

Pharmonics = $1/2\ \Sigma(an^2 + bn^2)$.

Experimental Procedure:

- Generate the periodic signal and compute Fourier coefficients.
- Calculate harmonic magnitude for n = 1 to 15.
- Convert harmonic number to frequency.
- Plot the harmonic spectrum using a stem plot.
- Identify the significant components.

Key Conditions & Threshold Values for Analysis:

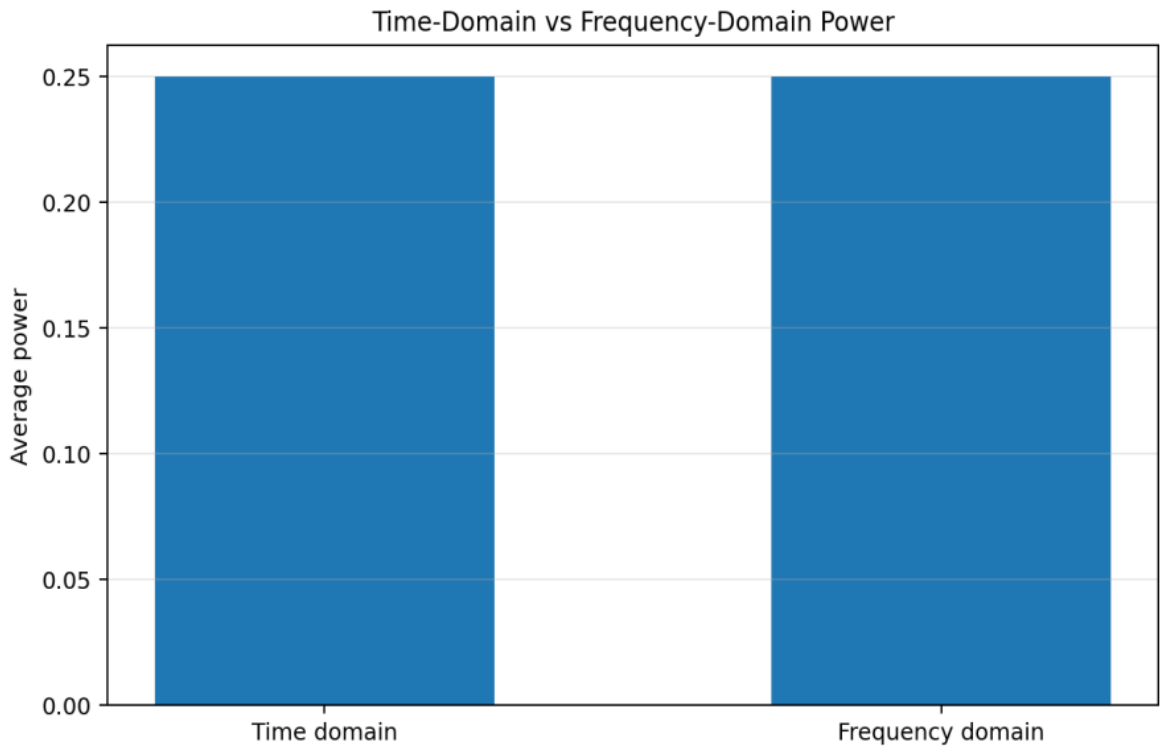
- Use f0 as the frequency spacing between adjacent harmonics.
- Use a stem plot for discrete harmonic components.

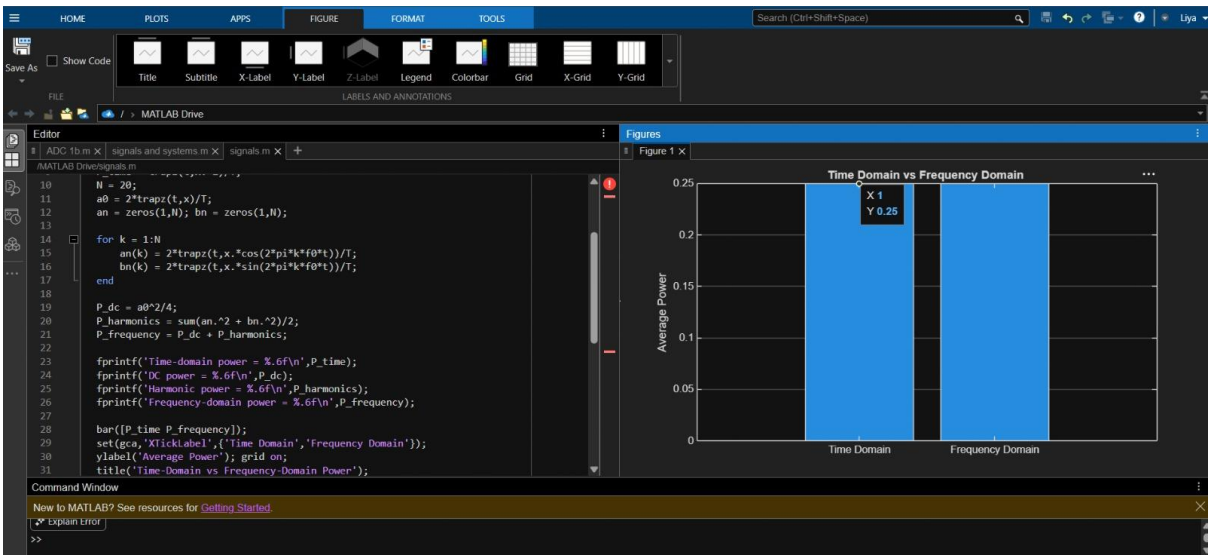
- Label the frequency axis in hertz.

MATLAB Coding:

```
clc; clear; close all;
% 6(iv) Time-Domain and Frequency-Domain Power
f0 = 1; T = 1/f0;
t = linspace(0,T,4000);
x = max(sin(2*pi*f0*t),0);
P_time = trapz(t,x.^2)/T;
N = 20;
a0 = 2*trapz(t,x)/T;
an = zeros(1,N); bn = zeros(1,N);
for k = 1:N
    an(k) = 2*trapz(t,x.*cos(2*pi*k*f0*t))/T;
    bn(k) = 2*trapz(t,x.*sin(2*pi*k*f0*t))/T;
end
P_dc = a0^2/4;
P_harmonics = sum(an.^2 + bn.^2)/2;
P_frequency = P_dc + P_harmonics;
fprintf('Time-domain power = %.6f\n',P_time);
fprintf('DC power = %.6f\n',P_dc);
fprintf('Harmonic power = %.6f\n',P_harmonics);
fprintf('Frequency-domain power = %.6f\n',P_frequency);
bar([P_time P_frequency]);
set(gca,'XTickLabel',{'Time Domain','Frequency Domain'});
ylabel('Average Power'); grid on;
title('Time-Domain vs Frequency-Domain Power');
```

Input and Output graphs:





Conclusion:

The experiment successfully verified Parseval's relation for continuous-time periodic signals by demonstrating the equivalence of power calculated in the time and frequency domains. The distribution of power among the fundamental and harmonic components was analyzed through the harmonic spectrum, and the total time-domain power was found to be consistent with the sum of harmonic powers in the frequency domain.

EXPERIMENT – 7:

CONTINUOUS-TIME FOURIER TRANSFORM AND SPECTRUM

(ii) To determine the magnitude and phase spectra of continuous-time signals and analyze their significant spectral components.

Expected Outcomes:

• Generate the Gaussian-modulated cosine signal.

• Compute its Fourier transform approximation.

• Plot magnitude and phase spectra.

• Identify the principal spectral peaks.

Software / Laboratory Environment:

MATLAB

S.No.	Output Parameter	Theoretical / Expected Value	Practical / Simulated Value	% Deviation / Acceptance
1	Time-domain signal	$e^{(-t^2)}\cos(2\pi 3t)$	Record plot	Qualitative
2	Magnitude spectrum	$\|X(f)\|$	Record plot	Qualitative
3	Phase spectrum	$\angle X(f)$	Record plot	Qualitative
4	Spectral peaks	Around ±3 Hz	Measure	≤5% where measurable

Formulae / Analysis:

$X(f) = \int x(t)e^{(-j2\pi ft)}dt$.

Magnitude spectrum = $|X(f)|$.

Phase spectrum = $\angle X(f)$.

Experimental Procedure:

- Define t and the sampling interval.
- Generate $x(t)=e^{(-t^2)}\cos(2\pi 3t)$.
- Compute the FFT-based transform with N = 4096.
- Plot the signal, magnitude and unwrapped phase.

- Identify the principal spectral peaks.

Key Conditions & Threshold Values for Analysis:

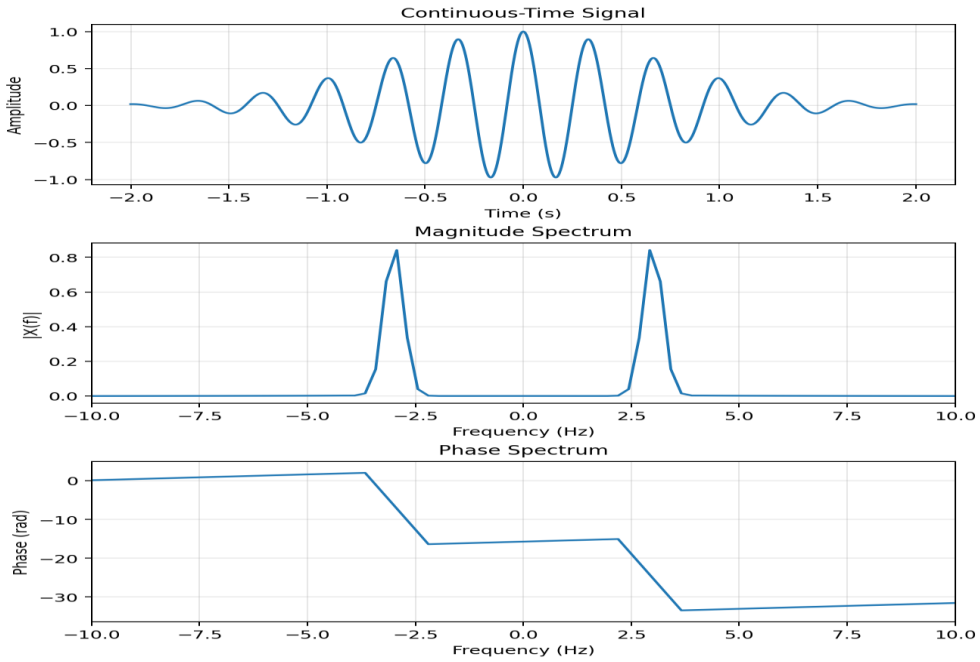
- Use the same time vector for signal and transform.
- Limit the displayed frequency range to the specified range for interpretation.
- Use phase unwrapping consistently.

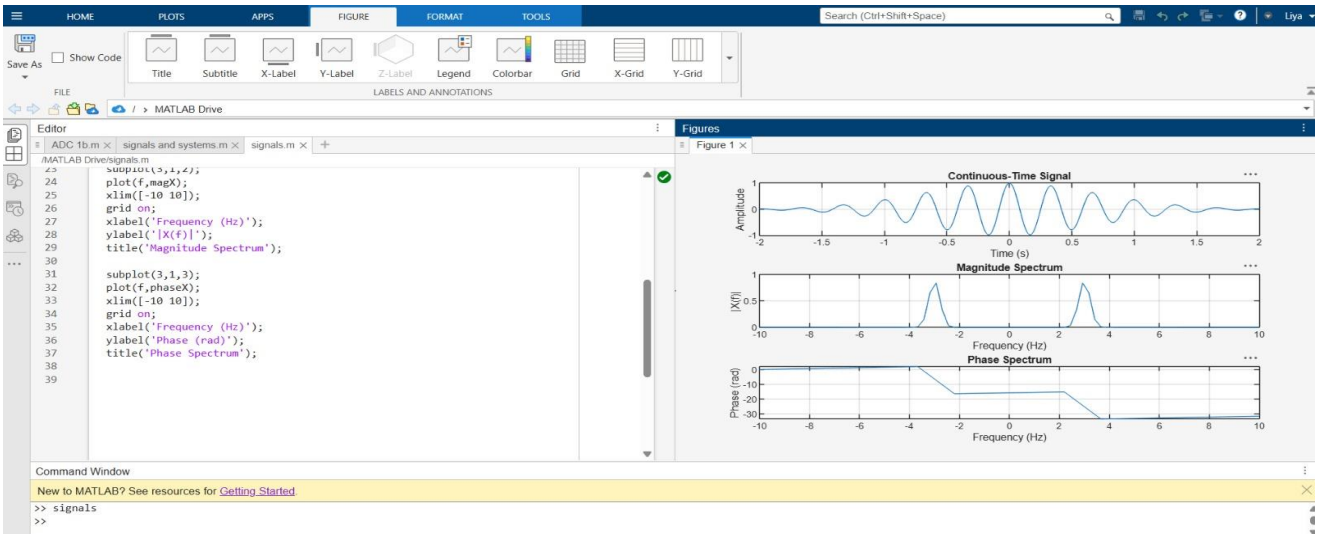
MATLAB Coding:

```
clc;
clear;
close all;
% 7(ii) Magnitude and Phase Spectra of Continuous-Time Signals
t = -2:0.001:2;
dt = t(2)-t(1);
x = exp(-t.^2).*cos(2*pi*3*t);
N = 4096;
X = fftshift(fft(x,N))*dt;
f = (-N/2:N/2-1)/(N*dt);
magX = abs(X);
phaseX = unwrap(angle(X));
figure;
subplot(3,1,1);
plot(t,x);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Continuous-Time Signal');
subplot(3,1,2);
plot(f,magX);
xlim([-10 10]);
grid on;
xlabel('Frequency (Hz)');
```

```
ylabel('|X(f)|');
title('Magnitude Spectrum');
subplot(3,1,3);
plot(f,phaseX);
xlim([-10 10]);
grid on;
xlabel('Frequency (Hz)');
ylabel('Phase (rad)');
title('Phase Spectrum');
```

Input and Output Graphs:





(iii) Verification of Fourier Transform Properties and Frequency-Domain Characteristics

Expected Outcomes:

• Generate a shifted version of a Gaussian signal.

• Compute the Fourier transform of the shifted signal.

• Calculate the theoretical shifted transform.

• Compare magnitude and phase responses.

Software / Laboratory Environment

MATLAB

Output Parameter Table S.No.	**Output Parameter**	**Theoretical / Expected Value**	**Practical / Simulated Value**	**% Deviation / Acceptance**
1	Shifted signal	x(t−0.4)	Record plot	Qualitative
2	Computed magnitude	|Xshift(f)|	Record plot	Compare
3	Theoretical transform	X(f)e^(−j2πft0)	Record plot	Compare
4	Phase change	−2πft0	Record plot	Compare

Formulae / Analysis:

Time-shift property: x(t−t0) ↔ X(f)e^(−j2πft0).

Magnitude is unchanged by an ideal time shift.

Phase changes linearly with frequency by −2πft0.

Experimental Procedure:

- Generate x(t)=e^(−t²).
- Create x(t−0.4) using interpolation.
- Compute the FFT of the shifted signal.
- Form the theoretical transform using the time-shift property.
- Compare computed and theoretical magnitude and phase.

Key Conditions & Threshold Values for Analysis:

- Use the same frequency axis for both transforms.
- Interpret phase using the same unwrap convention.
- Magnitude responses should be substantially coincident for an ideal shift.

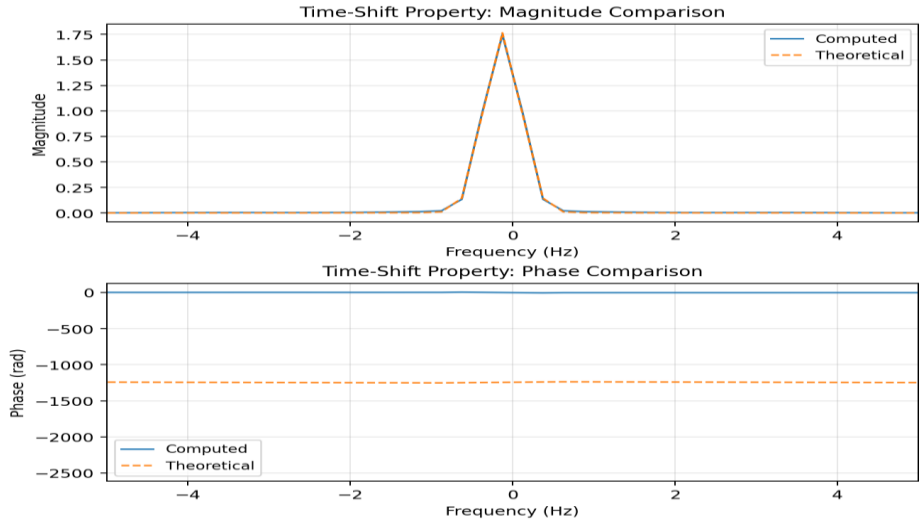
MATLAB Coding:

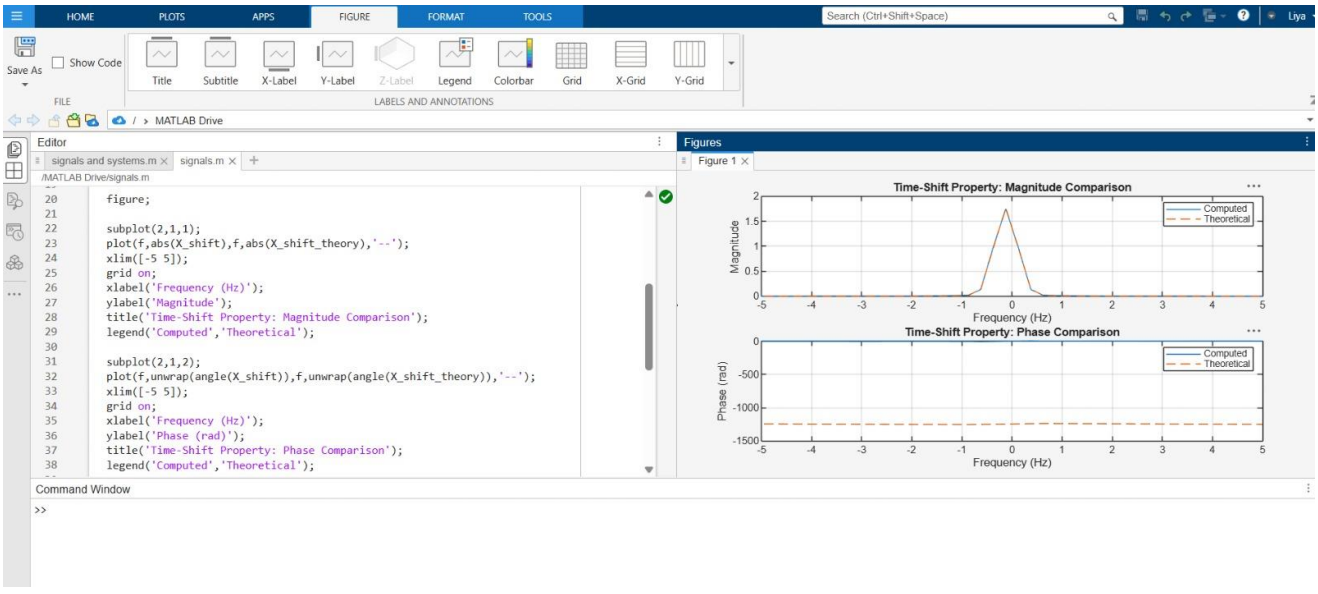
```
clc;
clear;
close all;
% 7(iii) Verification of Fourier Transform Properties
t = -2:0.001:2;
dt = t(2)-t(1);
x = exp(-t.^2);
t0 = 0.4;
x_shift = interp1(t,x,t-t0,'linear',0);
N = length(x);
f = (-N/2:N/2-1)/(N*dt);
X = fftshift(fft(x))*dt;
X_shift_theory = X.*exp(-1j*2*pi*f*t0);
X_shift = fftshift(fft(x_shift))*dt;
figure;
subplot(2,1,1);
plot(f,abs(X_shift),f,abs(X_shift_theory),'--');
xlim([-5 5]);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Time-Shift Property: Magnitude Comparison');
legend('Computed','Theoretical');
subplot(2,1,2);
plot(f,unwrap(angle(X_shift)),f,unwrap(angle(X_shift_theory)),'--');
xlim([-5 5]);
grid on;
xlabel('Frequency (Hz)');
ylabel('Phase (rad)');
```

title('Time-Shift Property: Phase Comparison');

legend('Computed','Theoretical');

Input and Output Graphs :





Conclusion:

The continuous-time Fourier transform of representative signals was successfully obtained using FFT-based approximation, and their magnitude and phase spectra were analyzed to identify significant frequency components. The time-shift property was verified by comparing the computed and theoretical Fourier transforms, confirming that time shifting preserves magnitude while introducing a frequency-dependent phase shift. The spectral components and occupied bandwidth of a composite communication signal were also determined, demonstrating the importance of frequency-domain analysis in communication-system design.

EXPERIMENT – 8:

LAPLACE TRANSFORM, INVERSE TRANSFORM, ROC AND STABILITY

OBJECTIVES:

i) To analyze the Laplace transform of continuous-time signals and systems and interpret their characteristics in the frequency domain.

ii) To determine the inverse Laplace transform of a given frequency-domain representation and reconstruct the corresponding time-domain signal.

iii) To determine the Region of Convergence (ROC) of Laplace transforms and analyze its significance in characterizing continuous-time signals and systems.

iv) To analyze system stability using poles, zeros, and the Region of Convergence (ROC) and verify the stability conditions of continuous-time systems.

(ii) Determination of Inverse Laplace Transform for Signal Reconstruction

Expected Outcomes:

• Obtain x(t) from X(s).

• Display the inverse-transform expression.

• Evaluate the reconstructed waveform numerically.

• Analyze the resulting time-domain response.

Software / Laboratory Environment:

MATLAB

Output Prameter Table S.No.	**Output Parameter**	**Theoretical / Expected Value**	**Practical / Simulated Value**	**% Deviation / Acceptance**
1	Inverse transform	e^(−2t)−e^(−3t)	Record result	Correct form
2	Initial value	0	Observe plot	Correct
3	Time response	x(t)	Record plot	Qualitative
4	Final value	Approaches 0	Observe plot	Qualitative

Input Parameter Table S.No.	Input Parameter	Symbol / Setting	Values
1	Laplace-domain signal X(s)	1/((s+2)(s+3))	—
2	Time range	0 to 5	s
3	Time step	0.001	s
4	Toolbox	Symbolic Math Toolbox	—

Formulae / Analysis:

X(s)=1/[(s+2)(s+3)].

Partial fractions: X(s)=1/(s+2)−1/(s+3).

x(t)=e^(−2t)−e^(−3t), t≥0.

Experimental Procedure:

- Define X(s) symbolically.
- Apply ilaplace to obtain x(t).
- Evaluate x(t) over 0≤t≤5 s.
- Plot the reconstructed time-domain signal.
- Compare the plotted response with the analytical expression.

Key Conditions & Threshold Values for Analysis:

• Use Symbolic Math Toolbox.

• Use the specified time interval and step.

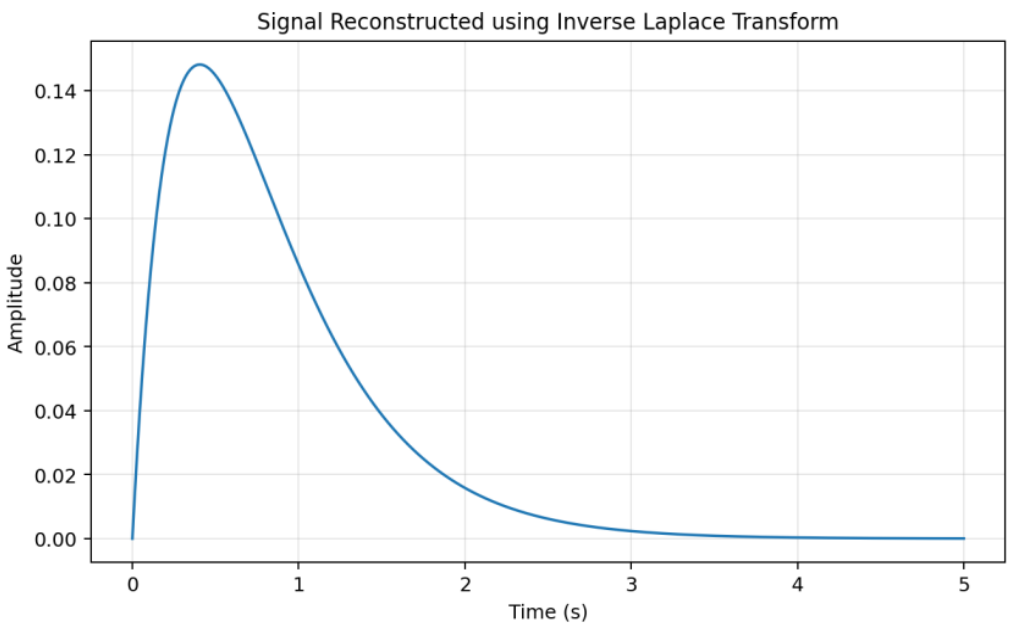
• The response should decay toward zero for the given poles.

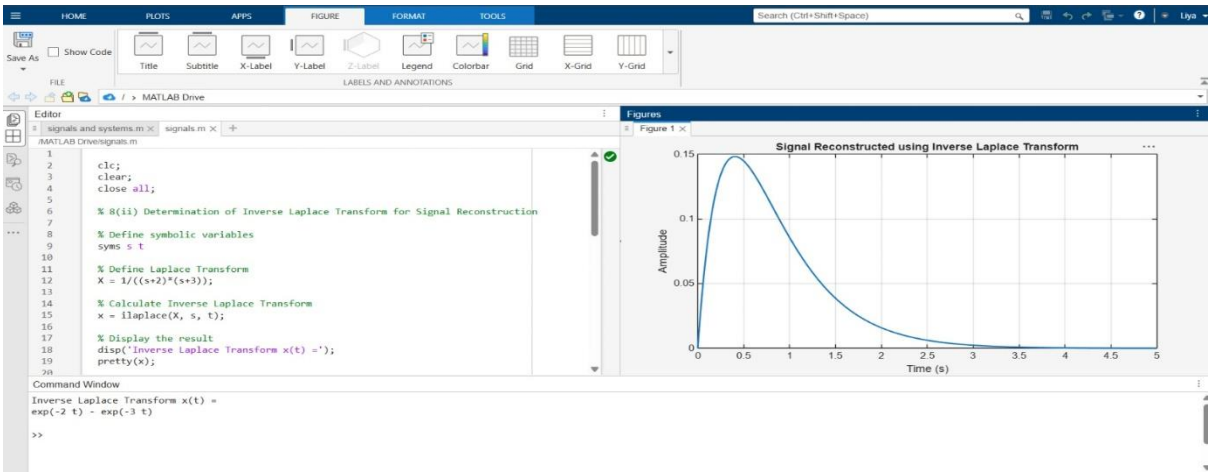
MATLAB Coding:

```
clc;
clear;
close all;
% 8(ii) Determination of Inverse Laplace Transform for Signal Reconstruction
% Define symbolic variables
syms s t
% Define Laplace Transform
X = 1/((s+2)*(s+3));
```

```
% Calculate Inverse Laplace Transform
x = ilaplace(X, s, t);
% Display the result
disp('Inverse Laplace Transform x(t) =');
pretty(x);
% Convert symbolic expression into a numerical function
tt = 0:0.001:5;
xx = double(subs(x, t, tt));
% Plot the reconstructed signal
figure;
plot(tt, xx, 'LineWidth', 1.5);
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
title('Signal Reconstructed using Inverse Laplace Transform');
xlim([0 5]);
```

Input and Output Graphs:





(iii) To determine the Region of Convergence (ROC) of Laplace transforms and analyze its significance in characterizing continuous-time signals and systems.

Expected Outcomes:

• Identify poles at s = −2 and s = −4.

• State the possible ROC cases.

• Relate each ROC to the corresponding signal sidedness.

• Interpret how ROC affects the system representation.

Software / Laboratory Environment:

MATLAB

Output Parameter Table S.No.	**Output Parameter**	**Theoretical / Expected Value**	**Practical / Simulated Value**	**% Deviation / Acceptance**
1	Pole locations	−2 and −4	Record plot	Correct
2	Right-sided ROC	Re(s)>−2	Record result	Correct
3	Left-sided ROC	Re(s)<−4	Record result	Correct
4	Two-sided ROC	−4<Re(s)<−2	Record result	Correct

Input Parameter Table S.No.	Input Parameter	Symbol / Setting	Value
1	Laplace transform	1/((s+2)(s+4))	—
2	Pole 1	−2	—
3	Pole 2	−4	—

Formulae / Analysis:

Poles occur at s = −2 and s = −4.

Right-sided ROC: Re(s) > −2.

Left-sided ROC: Re(s) < −4.

Two-sided ROC: −4 < Re(s) < −2.

Experimental Procedure:

- Define the pole locations.
- Plot the poles in the s-plane.
- Mark the vertical lines through the poles.
- State the candidate ROC for each signal-sidedness case.
- Interpret the relationship between poles and ROC.

Key Conditions & Threshold Values for Analysis:

• The ROC cannot include poles.

• Use the real part of s for the vertical ROC boundaries.

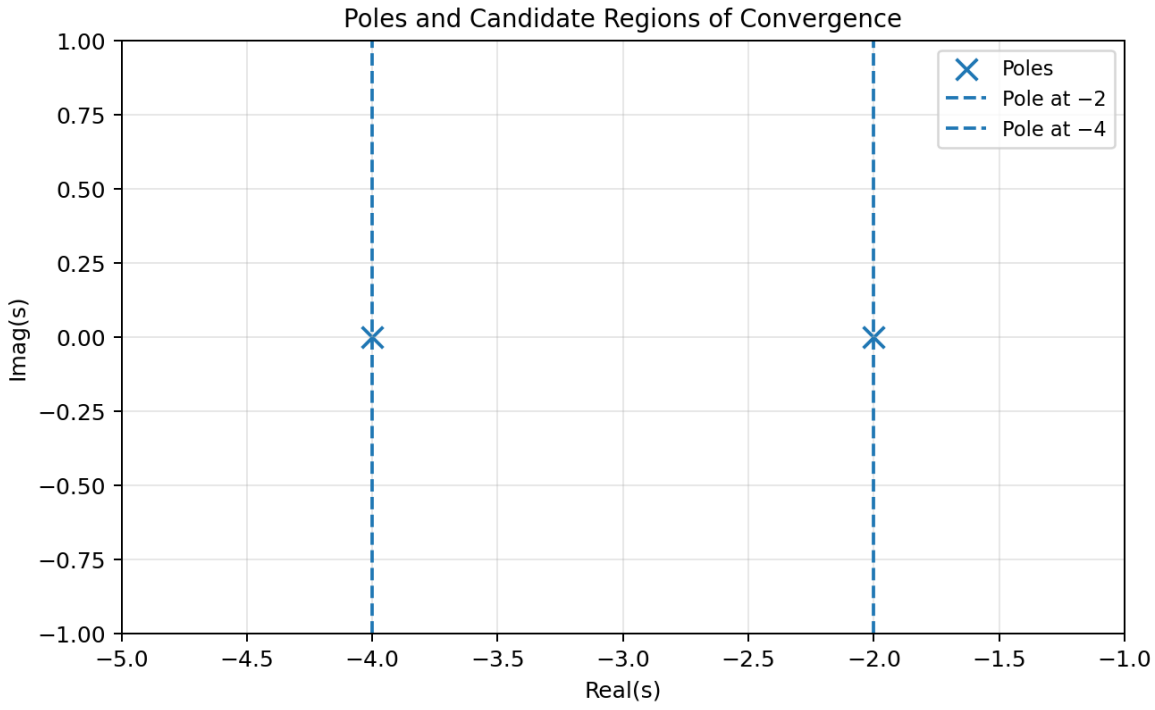
• Interpret right-, left- and two-sided cases separately.

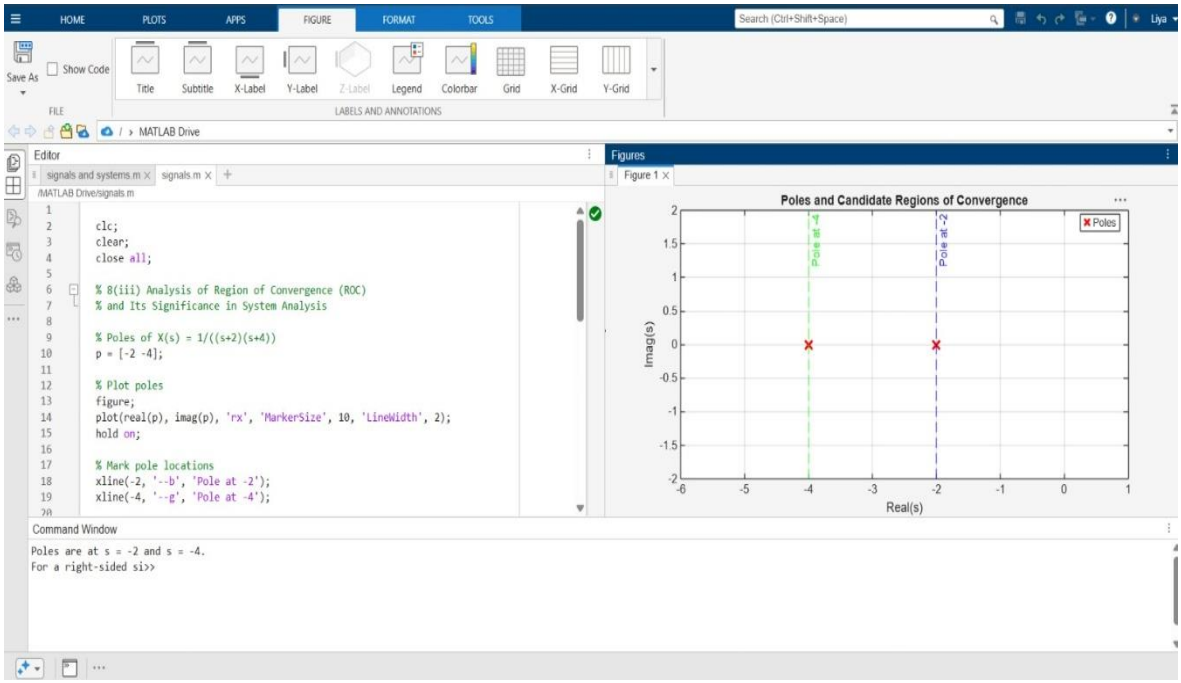
MATLAB Coding:

```
clc;
clear;
close all;
% 8(iii) Analysis of Region of Convergence (ROC)
% and Its Significance in System Analysis
% Poles of X(s) = 1/((s+2)(s+4))
p = [-2 -4];
% Plot poles
figure;
plot(real(p), imag(p), 'rx', 'MarkerSize', 10, 'LineWidth', 2);
```

```
hold on;
% Mark pole locations
xline(-2, '--b', 'Pole at -2');
xline(-4, '--g', 'Pole at -4');
% Graph settings
grid on;
xlabel('Real(s)');
ylabel('Imag(s)');
title('Poles and Candidate Regions of Convergence');
legend('Poles', 'Location', 'best');
xlim([-6 1]);
ylim([-2 2]);
% Display ROC information
fprintf('Poles are at s = -2 and s = -4.\n');
fprintf('For a right-sided si
```

Input and Output Graphs:




HOME
PLOTS
APPS
FIGURE
FORMAT
TOOLS
Search (Ctrl+Shift+Space)
Liya
Save As
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
MATLAB Drive
Editor
signals and systems.m
signals.m
/MATLAB Drive/signals.m
clc;
clear;
close all;
% 8(iii) Analysis of Region of Convergence (ROC)
% and Its Significance in System Analysis
% Poles of X(s) = 1/((s+2)(s+4))
p = [-2 -4];
% Plot poles
figure;
plot(real(p), imag(p), 'rx', 'MarkerSize', 10, 'LineWidth', 2);
hold on;
% Mark pole locations
xline(-2, '--b', 'Pole at -2');
xline(-4, '--g', 'Pole at -4');
Figures
Figure 1
Poles and Candidate Regions of Convergence
Pole at -4
Pole at -2
Poles
Imag(s)
Real(s)
Command Window
Poles are at s = -2 and s = -4.
For a right-sided si>>